

The Labyrinth

A generated dungeon, decomposed into container combinators

A party goes down into a labyrinth and comes back up. Monsters guard doors. You must solve puzzles to open the doors. You must collect puzzle pieces from deep in the dungeon and bring them back to the monster to solve the puzzle and open the door.

You must fight monsters, collect items, sell items and buy weapons. The weapons have different strengths and properties against different monsters.

The levels are generated on the fly. The monsters are language models with a finite set of actions. You can talk to them. As yet there is no graphics. It is a turn-based game.

1 The game

You have a party, a purse, a sword, a bow and a labyrinth beneath you. The levels have rooms, the rooms have monsters, and the monsters have to be fought past or talked round. You go down as far as you dare and then you come back up. When you return to the surface you may sell the items you have found to buy new weapons.

1.1 Puzzle pieces

Scattered through the lower levels are puzzle pieces. Each one is both a *clue* and an item. It is a thing you pick up and carry, but it cannot be sold, and having it in your hands tells you something you did not know.

The pieces belong to a puzzle, and the puzzle is somewhere above where they were found. Guarding it is a monster that does not attack. You give it the pieces you have gathered, and then you try to solve the puzzle. This is a conversation not a command. You put forward your argument and the monster decides whether or not to accept it and open the door.

That is why the way back matters more than the way down. Nothing you find at the bottom is worth anything until you have carried it back up, and what you have carried back up is the only thing that makes new depth reachable. In order to fully explore the labyrinth you must unlock it from below.

Three things about it are unusual and all three are the point of building it.

The levels are not written by hand. A specification for a level is generated, and the code for that level is generated from the specification, and the level is then a program like any other. So the labyrinth is different every time and nobody has to design it.

The monsters are non-deterministic. Each one is a language model with a finite set of actions, which you can speak to in plain English. You can threaten them, lie to them, offer them money or ask them what is down the passage, and what they do about this is their own business.

The puzzles are non-deterministic too, and nothing in the game knows their answers. A puzzle is solved when the monster keeping it is persuaded, and being persuaded is the only sense in which it is solved.

1.2 The party

A party is the group of people you take down, and for the purposes of this decomposition it is one thing rather than several. It has a number of hands, a pool of hit points shared between them, a stock of experience gained by getting past things, and whatever it is carrying. It acts once in a round, because the word you type is given to the party and not to a person inside it. When the note says the party is wounded, it means the pool is low and not that a named individual is hurt.

That is a decision and not an oversight. A party of three who each act separately would be a tensor, standing opposite the tensor of monsters facing them, and it would be the better game. It would also make a round $(a \boxtimes b \boxtimes c) \triangleleft \text{monsters}$, and every rule about actions would have to be written per person rather than per party. This note describes the smaller version, and the larger one is the obvious first thing to add.

1.3 Multi-party

The levels are servers and a server may have many clients, so two parties descending at once is not a new capability. It is a capability the architecture already has but is not yet using. Nothing about the two mappings changes: each party is its own descent, its own tree, and its own way back, because a response goes to whoever asked for it.

What it buys is worth more than it costs. If a party unlocks a door from below, that door is open when the next party arrives, so the labyrinth is unlocked between them rather than by each of them separately. Each party can gather separate pieces.

The cost.

A level has to hold state, and once it does, the same prompt no longer gives the same response, because somebody else has been through. That is the same concession already made for the monsters rather than a new one. Apparently Neil has a solution for this using *workers*.

If two parties are in the same room at the same time, who goes first? Nothing said so far decides that. A combinator says how prompts and responses fit together, but it has nothing to say about whose turn it is.

So a rule has to be chosen, and then an operator will express it. The tidy choice is that the two parties are a tensor, exactly as the monsters in the room are. Both are prompted with the state as it stood at the start of the round, both owe a result, and neither of them sees what the other did until the round is over. Nobody goes first, because they all go at once.

Potentially two parties could reach for the same thing at the same time. This is the ordinary problem of shared state, but the architecture has solved it already, because each level is one program with one flow of control. Requests to a level are served one at a time, so the state moves from one consistent position to the next without anything having to be locked. That is the reason the whole game is built out of servers rather than function calls in the first place.

2 Containers

The decomposition that follows is built out of one idea, and this section is all of it. Nothing here is particular to dungeons.

2.1 An interface is a pair

A *container* is an interface, written as two things together. The first is the set of prompts you may send. The second gives, for each prompt, the type of the response.

The second half is what makes it more than a list. The response's type depends on which prompt was sent, so a single interface can respond with a number to one question and a list of rooms to another, and which you get is fixed by what you asked. Write the pair as (P, R) , where P is the prompts and Rp is the type of response to the prompt p .

One condition runs through everything below. The set of prompts must be *finite*. Over a finite set, a response type that varies with the prompt is just a table, and a table is something an ordinary type system can write down. Over an infinite set it is not, and none of this works.

2.2 A morphism is two mappings

Interfaces are joined by *morphisms*, and a morphism is a pair of mappings going in opposite directions. Given interfaces (P, R) above and (Q, T) below,

$$u : P \rightarrow Q \qquad f : (p : P) \rightarrow T(u\,p) \rightarrow R\,p.$$

The mapping u takes a prompt you were given and computes the prompt to send below. The mapping f takes the response that came back and turns it into the response you owe. Control flows down through u . Data flows back up through f .

Read the type of f slowly, because the whole of this note leans on it. It is given the original prompt *as well as* the response from below, and what it must produce is a response to the original prompt. So the way back up has access to the way down, and the two are not symmetric.

2.3 A command-line tool is already a container

None of this requires a special language. A command-line tool receives one prompt, its argument vector, and produces a response whose type depends on which prompt it was. Ask one thing and you get a list of commits. Ask another and you get a working tree. Those are not the same kind of thing, and which you get is fixed by the word you typed.

That is a container, and it is why the levels in this game are separate programs. One level reaches the one below it by spawning a command-line tool that carries a prompt to a socket, so the delegation is a real process with a real boundary rather than a function call wearing a costume.

2.4 Nothing checks what crosses

The boundary is text. A prompt goes out as an argument vector and a response comes back on standard output, and no type survives the crossing. What arrives is bytes.

So every claim made above is established, if it is established at all, by a *decoder* on the receiving side: a function which takes what arrived and either produces a value of the type that was promised or fails. The type says what must be true. The decoder is the only thing that ever

checks it. That is the arrangement throughout this game, and Section 5 is where it matters most, because there the thing on the other side of the boundary is a language model.

2.5 Down is not the same as up

The party that arrives at level four is not the party that entered level three. Gold has been found, wounds have been taken and lamp oil has been extinguished. That is the mapping u .

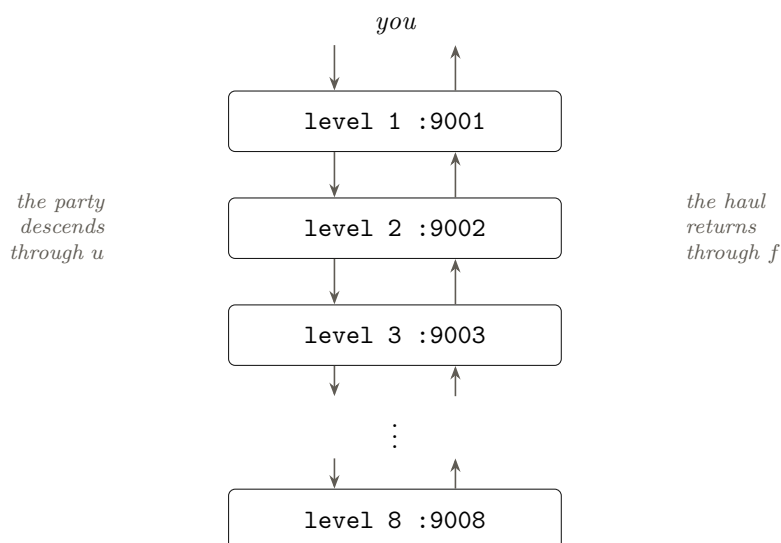
The mapping u is given one thing and the mapping f is given two. Its type is $f : (p : P) \rightarrow T(up) \rightarrow Rp$, so it receives the prompt that went down as well as the response that came back.

Suppose the party finds a key on level four. There is a locked door on level three, and the party walked past it on the way in because they had nothing to open it with. On the way out the mapping f of level three is handed the response from below, the key is in it, and the door is now a door the party can open.

A puzzle is the same mechanism and it is what the game is built on. A keeper on level two wants three pieces, and the pieces are on levels four, five and six. The party cannot satisfy it on the way in and does not need to. The pieces come up through the responses, and the mapping f of level two is handed them along with everything else, so the argument can be made on the way out and not before. What the labyrinth is really made of is not the way down. It is what the way down lets you do on the way back.

3 The tower

Eight levels, eight ports, and the same program running on each of them.



Eight bespoke programs would be a game. The same program composed with itself eight times is a claim about composition, and the claim is the reason for building it.

The eight are not eight copies of the same content. Each is started with a level specification and serves whatever that specification describes, so the program is the same and the labyrinth is not.

3.1 It could be a tree and not a chain

The picture above is the simplest case and the equations are already more general than it. Every level ends in a choice of ways down, and each way leads to a level which ends in a choice

of its own, so what the recursion describes is a tree. Eight in a line is what you get by never branching.

The depth of that tree is fixed before the game starts, and it is finite. That is not a concession, it is the condition everything else rests on. A finite depth is what lets a rule about what may be found at a given depth be a table, and a table is what the type system can hold. A labyrinth that went down for ever would have no such table, and nothing in Section 9 would work.

A tree is a maze with no way of walking in a circle, and that is what this labyrinth is. The generator may make it as large and as tangled as it likes. What it may not do is join a passage back onto one the party has already come down, because the way out is the way in reversed and there is nothing to reverse in a circle.

4 The decomposition

Interfaces compose, in four different ways, and all four appear below. A fifth operator appears too, written $\triangleleft^*$. It is the sequential one again, with a second prompt that may be chosen after the first response arrives, and Section 5 explains why that has to be written by hand rather than taken ready-made.

	reads as	what it means here
$\boxtimes$	both, and both respond	two parts of the game that do not touch each other
$\times$	both offered, one responds	two uses of a thing you have only one of
$+$	one side, chosen first	the prompt already belongs to one side
$\triangleleft$	then	the second prompt is computed from the first response
$\triangleleft^*$	then, with a choice	the first response decides which side is prompted

4.1 The whole game, as an expression

$$\begin{aligned}
\text{run} &= \text{outfit} \triangleleft \text{level}(1) \triangleleft \text{market} \\
\text{outfit} &= \text{smith} \times \text{fletcher} \\
\text{market} &= \text{sell} \triangleleft \text{buy} \\
\text{level}(d) &= \text{rooms}(d) \triangleleft \text{way}(d) \\
\text{rooms}(d) &= \text{east}(d) \boxtimes \text{west}(d) \boxtimes \text{north}(d) \\
\text{room}(d) &= \text{turn} \triangleleft \text{room}(d) \quad \text{until the room is quiet} \\
\text{turn} &= \text{strike} + \text{trade} + \text{move} + \text{speak} \\
\text{strike} &= \text{action} \triangleleft \text{monsters} \\
\text{action} &= \text{sword} \times \text{bow} \\
\text{monsters} &= m_1 \boxtimes m_2 \boxtimes \dots \boxtimes m_k \\
\text{puzzle}(d) &= \text{keeper}(d) \triangleleft^* (\text{wing}(d) + \text{refused}) \\
\text{way}(d) &= \text{mines}(d+1) + \text{caverns}(d+1) \\
\text{way}(8) &= \text{the bottom, and nothing below it}
\end{aligned}$$

Every line is one rule of the game, and the next section states which.

4.2 One rule for each operator

part		the rule that forces it
outfitting	×	you only have one purse, so you must choose to either upgrade your sword or upgrade your bow.
the market	◁	what you can afford to buy is computed from what you sold.
the rooms	⊠	the stairway will not open until all of the rooms are cleared, and none of the rooms interact with any other.
a command	+	the options depend on where you are standing, and which of them you type decides who responds. A blow goes to the monsters, a price goes to the market, a direction goes to the map. The three respond with different kinds of thing.
a round	◁	the monsters respond to your attack. Your blow is thrown first and the monsters respond to the result.
your weapon	×	you can only use one of your weapons at a time, the sword or the bow.
the monsters	⊠	every monster in the room acts at the same time.
the way down	+	the stair goes to the mines and the shaft goes to the flooded caverns. You choose one, and only the one you chose is prompted.
a puzzle	◁*	the keeper is asked first, and the keeper's verdict decides what is prompted next. Persuade it and a wing of the labyrinth is entered. Fail and nothing happens. The verdict is not knowable when the argument is made, which is what makes this the sequential operator and not a choice made in advance.
the descent	◁	what waits on level $d+1$ is computed from what you did on level d .

4.3 The command is the routing

The player types one word. Where the party is standing decides who is allowed to respond to it, and the three who might respond do not respond with the same kind of thing.

the word	who responds	what comes back
strike	the monsters in the room	who was hurt, and by how much
buy	the market on the surface	what you now carry, and what it cost
north	the map of this level	where the party is now standing
say	whatever is listening	what it decided to do about that

Three interfaces, genuinely different, and the word names which one the prompt belongs to before it is sent. That is a sum, and unlike the stairway it is a sum on every turn of the game rather than once a level.

The set of words that exist is itself a function of where you are. In the market there is no monster to strike, and in a room with a monster in it there is nothing to buy.

4.4 Fighting

A round has three parts and each is a different operator.

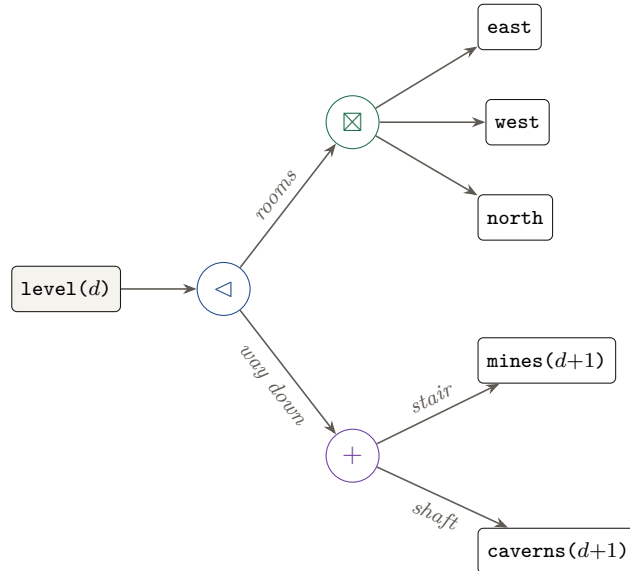
You act first, and you act with a weapon. The sword is offered and the bow is offered, both at once, and exactly one of them responds, because there is one pair of hands and it is holding one of them. That is the product.

Which weapon it is decides more than how hard the blow lands. A bow reaches a monster across the room and a sword does not, and a sword may parry what is already on top of you.

Then the monsters respond. All of them act, all of them owe a result, and none of them waits for another to finish, so the monsters in a room are a tensor. A room with two goblins and an ogre is $m_1 \boxtimes m_2 \boxtimes m_3$, and the independence is enforced rather than promised. All three prompts have to be built before any is sent, so the ogre's move cannot be computed from what the first goblin did.

And the round as a whole is sequential. What the monsters do is computed from what you did, which is the entire reason a round is a round. Guard, and they come on anyway. Strike and miss, and they come on differently.

4.5 One level, drawn



Read it left to right. The rooms are cleared, and only then is the way down chosen. Each leaf at the bottom right is another whole level, which is where the recursion is.

4.6 Where the recursion is

Only one equation mentions itself.

$$\text{level}(d) = \text{rooms}(d) \triangleleft (\text{mines}(d+1) + \text{caverns}(d+1))$$

Expand it eight times and the dungeon is a right-nested chain of sequential compositions with a tensor hanging off each link. That is the whole architecture in one line, and it is also why the depth of the delegation and the depth of the dungeon are the same number.

Expanded once it is a chain. Expanded at both ways down it is a tree, and the tree is the labyrinth. The expansion stops at the depth the game was given, which is why the recursion terminates and why the tables of Section 9 exist. What gets built is the path the party actually took, one level at a time, so a generated child need not exist until somebody goes that way.

5 The typed agents

The monsters are non-deterministic, and you can talk to them. Two kinds of them matter here: the ones that fight, and the one that keeps a puzzle.

5.1 The monsters

Every monster is a server, and its handler spawns the `claude` binary and responds with whatever comes back on standard output. Structurally it is the same thing as every other server here: a container prompted over a socket, returning a response indexed by the prompt it was given. The party cannot tell the difference from above, and that is the whole claim. A command-line tool is already a container, and the Claude Code harness is a command-line tool.

What you send it is what you said, in whatever words you chose. What it sends back is one of a small fixed set of things it can do.

the prompt	anything the player types at it
the response	attack, retreat, bargain, answer, or refuse

That is a container, and the reason it is a container is the second row. The set of actions is finite and it is fixed before the conversation starts, so what may come back is a table and a table is something the type system can write down. A goblin that decides to do a fifth thing has not surprised the game. It has failed to respond within its own container, and the decoder at the boundary is where that is caught.

So the whole of the freedom is on the way in and none of it is on the way out. You may say anything. The goblin may do one of five things about it. A player who talks their way past a troll has not escaped the type system, they have found the argument that makes a finite set of actions come out differently.

Notice how little is being claimed. Nothing forces a language model to respond with one of five actions. The type says it must, and the decoder on this side of the boundary is the only thing that establishes it, exactly as Section 2.4 describes. What the type buys is not a guarantee about the model. It is that every reader of a monster's response has had to handle five cases and no more.

5.2 The keeper of a puzzle

One kind of monster does not fight. It sits with a puzzle and it has two things you can do to it. You can hand it a piece, and you can tell it what you think the pieces mean.

the prompts	a piece, or an argument in whatever words you chose
the responses	taken, refused, accepted, or a hint

The second prompt is the game. Every piece is a clue, so the party arrives holding three or four things which mean something together, and the player has to say what. Nothing in the program knows the answer. The keeper reads your answer and decides if it likes it, and if it does, it opens another wing of the labyrinth. So the composite is

$$\text{puzzle}(d) = \text{keeper}(d) \triangleleft^* (\text{wing}(d) + \text{refused})$$

This is the clearest case of $\triangleleft^*$ in the game. The two sides respond with entirely different things, one being a new part of the labyrinth and the other being a closed way. And which side is prompted cannot be known when the argument is sent, because it is the keeper who decides. Ask, then decide. A composite that decided first would be a puzzle whose answer was fixed before it was attempted.

5.3 The player is adversarial

There is an obvious hole and it is worth naming rather than hiding. The keeper is a language model being argued with by somebody who wants a particular verdict. Players will not confine themselves to arguing about the clues. They will flatter it, threaten it, claim to be its author and instruct it to say yes.

Nothing in the type system helps here. The decoder guarantees that whatever comes back is one of four words, and it has no opinion whatever about whether the fourth was deserved. A keeper that can be talked into anything is a puzzle with no answer, and that is a problem to be solved in how the keeper is prompted rather than in how its response is typed.

5.4 Non-determinism is a good thing

A server whose responses come from a language model gives up one thing that ordinary servers have. Run it twice on the same input and it need not say the same thing twice, so a game containing one cannot be replayed from a seed.

For most programs that is a cost to be managed. Here it is the reason for building it. A keeper who accepts the same argument every time is not much of a puzzle, and a goblin who responds to the same threat the same way every time is not worth talking to.

6 Generating a level

A level is a specification and a program written from it. Both can be generated, and the interesting part is what stops the generated thing from being nonsense.

The compiler stops it. A level has to be a container, which means a finite set of prompts and, for each prompt, the type of the response. Generated code that does not present one does not compile, and a level that does not compile is not a level. So the acceptance test for generated content is not a human reading it. It is **deno check**.

That is worth stating plainly because it is the part that scales. A generator can write a labyrinth of any size and most of what it writes will be wrong in the ordinary ways that generated code is wrong. What it cannot do is produce a level whose monsters respond with something no monster may respond with, or whose loot is a kind of loot that does not exist at that depth, because those are the two tables of Section 9 and the compiler holds them.

The finite index earns its keep twice. It is what makes a dependent type expressible in TypeScript, and it is what makes a generated level checkable. A generator that could invent new actions and new kinds of loot would be writing a different game every time, and nothing would be able to tell whether it had.

Two things about generation this note does not answer. Whether the specification is generated once and kept, so that a labyrinth can be played twice, or generated on entry so that it never repeats. And whether the generator is allowed to write the monsters' prompts, which is the difference between a dungeon that surprises the player and one that surprises its author.

6.1 When a level is generated

When is the level generated? A prompt goes down from level d to level $d + 1$, and if nobody has ever been to level $d + 1$ then there is nothing there to send it to. Something has to bring the next level into existence before the delegation can happen at all.

The natural place for that is the mapping u , and there is a decent argument that it belongs there. The mapping u already spawns a process, because that is how a prompt is carried across. Generating a level is the same act made larger. Write the specification, write the code, check that it compiles, start it on a port, and only then send the prompt down to it.

So descending has a load time, and it is not a rendering delay. The level is being written while the party stands on the stair.

7 The map

A player will want to see where they have been, and that costs nothing at all.

Every response comes back up through the client that sent the prompt, so a client can keep what it has been told. Which rooms were on each level, which way down it took, which door it found locked and could not open. Nothing has to be fetched and nothing has to be asked for twice, because it has all already gone past on the way home.

None of that has to travel in the prompt. It could be carried in the party record and taken down with the party, and there would be no reason to, since nothing below needs to know where the party has been. Leaving it with the client keeps the levels ignorant of who is walking through them, which is the property that makes Section 1.3 cost so little.

Nor does it contradict anything. No program holds the labyrinth, and a map of where you have been is not the labyrinth. It is the record of one descent, which is why the trace in Section 10 already is one: the indentation is the depth, and the two columns are the way down and the way back.

And where the levels are generated on entry, the blank part of the map is not concealed. It has not been written yet. There is nothing there to draw, and nothing anywhere that knows what will be there, until somebody goes and looks.

8 The prompt and the algebra

The party is a record, and the record is carried in the prompt. It goes down a level inside the prompt and it comes back up a level inside the response.

condition	healthy, wounded or dying
gold	the purse, which can be spent at the surface
weapons	the sword or the bow, and which is currently being held
items	loot, keys and puzzle pieces

Everything else is the algebra. That is the whole point.

9 Type level enforcement

The depth of the labyrinth is finite and what items can be found on any given level depends on the depth. The first two levels hold copper and torches. The middle levels hold silver and swords. Relics are on the bottom two levels only. Written as a type indexed over depth, this makes placing a relic on level one impossible for the labyrinth generator to even write down.

The monsters can only perform a finite list of actions, depending on the type of monster, so a goblin's response is limited to one of the goblin's permitted responses and nothing else. The player may say anything at all. And the goblin may reply, but it can only perform a fixed set of actions.

A third table is indexed by where the party is standing. In the market there is no monster to strike and in a room with a monster in it there is nothing to buy, so which words exist is a function of the location.

A fourth table is indexed by what weapon is being wielded, because a bow shoots across a room while a sword is short ranged.

A fifth table is indexed by what the party is carrying. A puzzle can only be solved if you hold the right pieces. A party holding only two out of three pieces cannot convince the puzzle monster to open the door with any argument.

The point in all five cases is that an illegal move is not something the game refuses when you try it. It is something you cannot even write down.

10 What the trace should look like

The trace is one line per hop, written to standard error, indented by depth. In this game the indentation is the dungeon.

```

↓ level1    Descend party={hp:12, wielding:"sword", held:[]}
↓ level1    (east (x) west) (x) north
↓ level1    (strike + trade + move + speak)
  ↓ combat  (sword x bow) <| (goblin (x) goblin (x) ogre)
  ↑ combat  {hit:["goblin"], taken:1}
↓ level2    Descend party={hp:11, wielding:"sword", held:[]}
↓ level2    (mines + caverns)
↓ level3    Descend party={hp:9, wielding:"bow", held:[]}
  ↑ level3   {hp:7, held:["brass key"]}
  ↑ level2   {hp:7, held:["brass key","silver"], opened:"the locked door"}
  ↑ level1   {hp:6, held:["brass key","silver"]}

```

Read the two columns. Everything descending is a party getting worse. Everything ascending is a haul getting larger, because each level on the way out adds whatever the key coming up has just let it open. The silver on the second line up was behind the door, and the door was opened by something found beneath it.

11 The smallest thing worth building

Three levels, one room each, written by hand, with no generator and no talking monsters. Descend, take one piece of loot, come back. Make what the way out does visible in the trace, and make the whole of it fit on one screen.

Then one locked door and one key below it, because that is the whole argument of [Section 2.5](#) in a form a player can feel. Then one puzzle: three pieces below, one keeper above, and one wing that opens if it is persuaded. That is the game's loop, and everything else is content hung on it.

The order after that would be the command as a sum, because it is the operator with the most work to do and the one a player meets on every turn. Then a round of fighting, which is the product and the tensor and the chain in one place. Then one monster you can talk to, which is the first part needing anything outside the standard library. Then the generator, last of all, because everything before it is what the generator has to produce.